MySQL Mini-Capstone Project

The 8 Capstone Questions

Cristine Angelica Marie Serrano Tolentino | DASP - B02

Dataset: Brazilian E-Commerce Public Dataset by Olist (Kaggle), loaded into MySQL 8.0 across six related tables and queried in MySQL Workbench.

All revenue figures count delivered orders only. Customer-level questions group on customer_unique_id rather than customer_id, because the dataset issues a new customer_id for every order.

Question 1: Who are the top 10 customers by total amount spent?

```sql
-- QUESTION 1: Who are the top 10 customers by total amount spent?

SELECT
    c.customer_unique_id,
    MAX(c.customer_city) AS Customer_City,
    MAX(c.customer_state) AS Customer_State,
    COUNT(DISTINCT o.order_id) AS Total_Orders,
    ROUND(SUM(op.payment_value), 2) AS Total_Spent
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_payments op
    ON o.order_id = op.order_id
WHERE o.order_status = 'delivered'
GROUP BY c.customer_unique_id
ORDER BY Total_Spent DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customer_unique_id	Customer_City	Customer_State	Total_Orders	Total_Spent
0a0a92112bd4c708ca5fde585afaa872	rio de janeiro	RJ	1	13664.08
da122df9eeddfedc1dc1f5349a1a690c	araruama	RJ	2	7571.63
763c8b1c9c68a0229c42c9fc6f662b93	vila velha	ES	1	7274.88
dc4802a71eae9be1dd28f5d788ceb526	campo grande	MS	1	6929.31
459bef486812aa25204be022145caa62	vitoria	ES	1	6922.21
ff4159b92c40ebe40454e3e6a7c35ed6	marilia	SP	1	6726.66
4007669dec559734d6f53e029e360987	divinopolis	MG	1	6081.54
eebb5dda148d3893cdaf5b5ca3040ccb	maua	SP	1	4764.34
48e1ac109decbb87765a3eade6854098	joao pessoa	PB	1	4681.78
c8460e4251689ba205045f3ea17884a1	porto alegre	RS	4	4655.91

Question 2: What is the monthly revenue trend across the dataset?

```sql
-- QUESTION 2: What is the monthly revenue trend across the dataset?
SELECT
    DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m') AS sales_month,
    COUNT(DISTINCT o.order_id) AS total_orders,
    ROUND(SUM(op.payment_value), 2) AS total_revenue,
    ROUND(SUM(op.payment_value) / COUNT(DISTINCT o.order_id), 2) AS avg_order_value
FROM orders o
JOIN order_payments op
    ON o.order_id = op.order_id
WHERE o.order_status = 'delivered'
GROUP BY sales_month
ORDER BY sales_month ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

sales_month	total_orders	total_revenue	avg_order_value
2016-10	265	46566.71	175.72
2016-12	1	19.62	19.62
2017-01	750	127545.67	170.06
2017-02	1653	271298.65	164.13
2017-03	2546	414369.39	162.75
2017-04	2303	390952.18	169.76
2017-05	3546	567066.73	159.92
2017-06	3135	490225.60	156.37
2017-07	3872	566403.93	146.28
2017-08	4193	646000.61	154.07
2017-09	4150	701169.99	168.96
2017-10	4478	751140.27	167.74
2017-11	7289	1153528.05	158.26
2017-12	5513	843199.17	152.95
2018-01	7069	1078606.86	152.58
2018-02	6555	966510.88	147.45
2018-03	7003	1120678.00	160.03
2018-04	6798	1132933.95	166.66
2018-05	6749	1128836.69	167.26
2018-06	6099	1012090.68	165.94
2018-07	6159	1027903.86	166.89
2018-08	6351	985414.28	155.16

Question 3: What is the month-over-month change in revenue?

Query

```sql
-- QUESTION 3: What is the month-over-month change in revenue?

WITH Monthly_Revenue AS (
    SELECT
        DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m') AS sales_month,
        ROUND(SUM(op.payment_value), 2) AS total_revenue
    FROM orders o
    JOIN order_payments op
        ON o.order_id = op.order_id
    WHERE o.order_status = 'delivered'
        AND o.order_purchase_timestamp >= '2017-01-01'
    GROUP BY sales_month
)
SELECT
    sales_month,
    total_revenue,
    LAG(total_revenue) OVER (ORDER BY sales_month) AS previous_month_revenue,
    ROUND(
        total_revenue - LAG(total_revenue) OVER (ORDER BY sales_month), 2
    ) AS change_in_revenue,
    ROUND(
        (total_revenue - LAG(total_revenue) OVER (ORDER BY sales_month)) * 100.0
        / LAG(total_revenue) OVER (ORDER BY sales_month), 2
    ) AS percent_change
FROM Monthly_Revenue
ORDER BY sales_month ASC;
```

Result

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

sales_month	total_revenue	previous_month_revenue	change_in_revenue	percent_change
2017-01	127545.67	NULL	NULL	NULL
2017-02	271298.65	127545.67	143752.98	112.71
2017-03	414369.39	271298.65	143070.74	52.74
2017-04	390952.18	414369.39	-23417.21	-5.65
2017-05	567066.73	390952.18	176114.55	45.05
2017-06	490225.60	567066.73	-76841.13	-13.55
2017-07	566403.93	490225.60	76178.33	15.54
2017-08	646000.61	566403.93	79596.68	14.05
2017-09	701169.99	646000.61	55169.38	8.54
2017-10	751140.27	701169.99	49970.28	7.13
2017-11	1153528.05	751140.27	402387.78	53.57
2017-12	843199.17	1153528.05	-310328.88	-26.90
2018-01	1078606.86	843199.17	235407.69	27.92
2018-02	966510.88	1078606.86	-112095.98	-10.39
2018-03	1120678.00	966510.88	154167.12	15.95
2018-04	1132933.95	1120678.00	12255.95	1.09
2018-05	1128836.69	1132933.95	-4097.26	-0.36
2018-06	1012090.68	1128836.69	-116746.01	-10.34
2018-07	1027903.86	1012090.68	15813.18	1.56
2018-08	985414.28	1027903.86	-42489.58	-4.13

Question 4: Which product categories generate the most revenue?

```sql
-- QUESTION 4: Which product categories generate the most revenue?

SELECT
    COALESCE(p.product_category_name, 'Uncategorized') AS Product_Category,
    COUNT(DISTINCT oi.order_id) AS Total_Orders,
    ROUND(SUM(oi.price), 2) AS Total_Revenue,
    ROUND(SUM(oi.price) / COUNT(DISTINCT oi.order_id), 2) AS Revenue_Per_Order
FROM products p
JOIN order_items oi
    ON p.product_id = oi.product_id
JOIN orders o
    ON oi.order_id = o.order_id
WHERE o.order_status = 'delivered'
GROUP BY Product_Category
ORDER BY Total_Revenue DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Product_Category	Total_Orders	Total_Revenue	Revenue_Per_Order
beleza_saude	8647	1233131.72	142.61
relogios_presentes	5495	1166176.98	212.23
cama_mesa_banho	9272	1023434.76	110.38
esporte_lazer	7530	954852.55	126.81
informatica_acessorios	6530	888724.61	136.10
moveis_decoracao	6307	711927.69	112.88
utilidades_domesticas	5743	615628.69	107.20
cool_stuff	3559	610204.10	171.45
automotivo	3810	578966.65	151.96
brinquedos	3804	471286.48	123.89

Question 5: Rank the top 3 products within each category by revenue.

Query, part 1

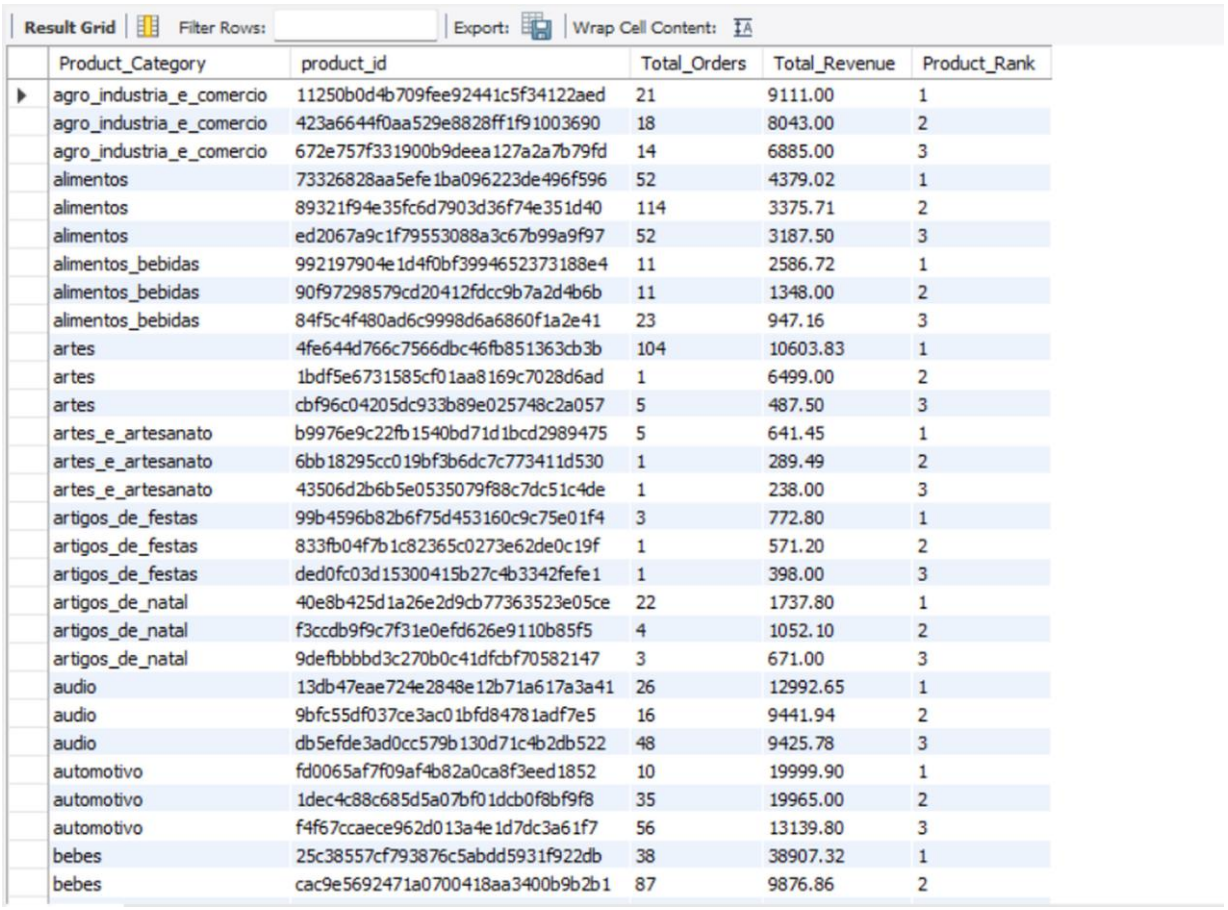
```sql
-- QUESTION 5: Rank the top 3 products within each category by revenue.

WITH Product_Revenue AS (
    SELECT
        COALESCE(p.product_category_name, 'Uncategorized') AS Product_Category,
        oi.product_id,
        COUNT(DISTINCT oi.order_id) AS Total_Orders,
        ROUND(SUM(oi.price), 2) AS Total_Revenue
    FROM products p
    JOIN order_items oi
        ON p.product_id = oi.product_id
    JOIN orders o
        ON oi.order_id = o.order_id
    WHERE o.order_status = 'delivered'
    GROUP BY Product_Category, oi.product_id
),
Ranked_Products AS (
    SELECT
        Product_Category,
        product_id,
```

Query, part 2

```sql
    GROUP BY Product_Category, oi.product_id
),
Ranked_Products AS (
    SELECT
        Product_Category,
        product_id,
        Total_Orders,
        Total_Revenue,
        DENSE_RANK() OVER (
            PARTITION BY Product_Category
            ORDER BY Total_Revenue DESC
        ) AS Product_Rank
    FROM Product_Revenue
)
SELECT
    Product_Category,
    product_id,
    Total_Orders,
    Total_Revenue,
    Product_Rank
FROM Ranked_Products
WHERE Product_Rank <= 3
ORDER BY Product_Category ASC, Product_Rank ASC;
```

Result



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Product_Category	product_id	Total_Orders	Total_Revenue	Product_Rank
agro_industria_e_comercio	11250b0d4b709fee92441c5f34122aed	21	9111.00	1
agro_industria_e_comercio	423a6644f0aa529e8828ff1f91003690	18	8043.00	2
agro_industria_e_comercio	672e757f331900b9deea127a2a7b79fd	14	6885.00	3
alimentos	73326828aa5efe1ba096223de496f596	52	4379.02	1
alimentos	89321f94e35fc6d7903d36f74e351d40	114	3375.71	2
alimentos	ed2067a9c1f79553088a3c67b99a9f97	52	3187.50	3
alimentos_bebidas	992197904e1d4f0bf3994652373188e4	11	2586.72	1
alimentos_bebidas	90f97298579cd20412fdcc9b7a2d4b6b	11	1348.00	2
alimentos_bebidas	84f5c4f480ad6c9998d6a6860f1a2e41	23	947.16	3
artes	4fe644d766c7566dbc46fb851363cb3b	104	10603.83	1
artes	1bdf5e6731585cf01aa8169c7028d6ad	1	6499.00	2
artes	cbf96c04205dc933b89e025748c2a057	5	487.50	3
artes_e_artesanato	b9976e9c22fb1540bd71d1bcd2989475	5	641.45	1
artes_e_artesanato	6bb18295cc019bf3b6dc7c773411d530	1	289.49	2
artes_e_artesanato	43506d2b6b5e0535079f88c7dc51c4de	1	238.00	3
artigos_de_festas	99b4596b82b6f75d453160c9c75e01f4	3	772.80	1
artigos_de_festas	833fb04f7b1c82365c0273e62de0c19f	1	571.20	2
artigos_de_festas	ded0fc03d15300415b27c4b3342fefe1	1	398.00	3
artigos_de_natal	40e8b425d1a26e2d9cb77363523e05ce	22	1737.80	1
artigos_de_natal	f3ccdb9f9c7f31e0efd626e9110b85f5	4	1052.10	2
artigos_de_natal	9defbbbbd3c270b0c41dfcbf70582147	3	671.00	3
audio	13db47eae724e2848e12b71a617a3a41	26	12992.65	1
audio	9bfc55df037ce3ac01bfd84781adf7e5	16	9441.94	2
audio	db5efde3ad0cc579b130d71c4b2db522	48	9425.78	3
automotivo	fd0065af7f09af4b82a0ca8f3eed1852	10	19999.90	1
automotivo	1dec4c88c685d5a07bf01dcb0f8bf9f8	35	19965.00	2
automotivo	f4f67ccaece962d013a4e1d7dc3a61f7	56	13139.80	3
bebes	25c38557cf793876c5abdd5931f922db	38	38907.32	1
bebes	cac9e5692471a0700418aa3400b9b2b1	87	9876.86	2

Question 6: Segment customers into spend tiers (Low / Medium / High).

```sql
-- QUESTION 6: Segment customers into spend tiers (Low / Medium / High).

SELECT
    Customer_Value_Tier,
    COUNT(*) AS Total_Customers,
    ROUND(SUM(Lifetime_Spend), 2) AS Tier_Revenue,
    ROUND(AVG(Lifetime_Spend), 2) AS Average_Spend,
    ROUND(SUM(Lifetime_Spend) * 100.0 / SUM(SUM(Lifetime_Spend)) OVER (), 2) AS Percent_Of_Revenue
FROM (
    SELECT
        c.customer_unique_id,
        SUM(op.payment_value) AS Lifetime_Spend,
        CASE
            WHEN SUM(op.payment_value) >= 500 THEN 'High Value'
            WHEN SUM(op.payment_value) >= 150 THEN 'Medium Value'
            ELSE 'Low Value'
        END AS Customer_Value_Tier
    FROM customers c
    JOIN orders o
        ON c.customer_id = o.customer_id
    JOIN order_payments op
        ON o.order_id = op.order_id
    WHERE o.order_status = 'delivered'
    GROUP BY c.customer_unique_id
) AS Customer_Spend
GROUP BY Customer_Value_Tier
ORDER BY Tier_Revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Customer_Value_Tier	Total_Customers	Tier_Revenue	Average_Spend	Percent_Of_Revenue
Medium Value	27288	6584565.87	241.30	42.69
Low Value	61804	4900257.61	79.29	31.77
High Value	4265	3937638.29	923.24	25.53

Question 7: How many customers are repeat buyers vs one-time buyers?

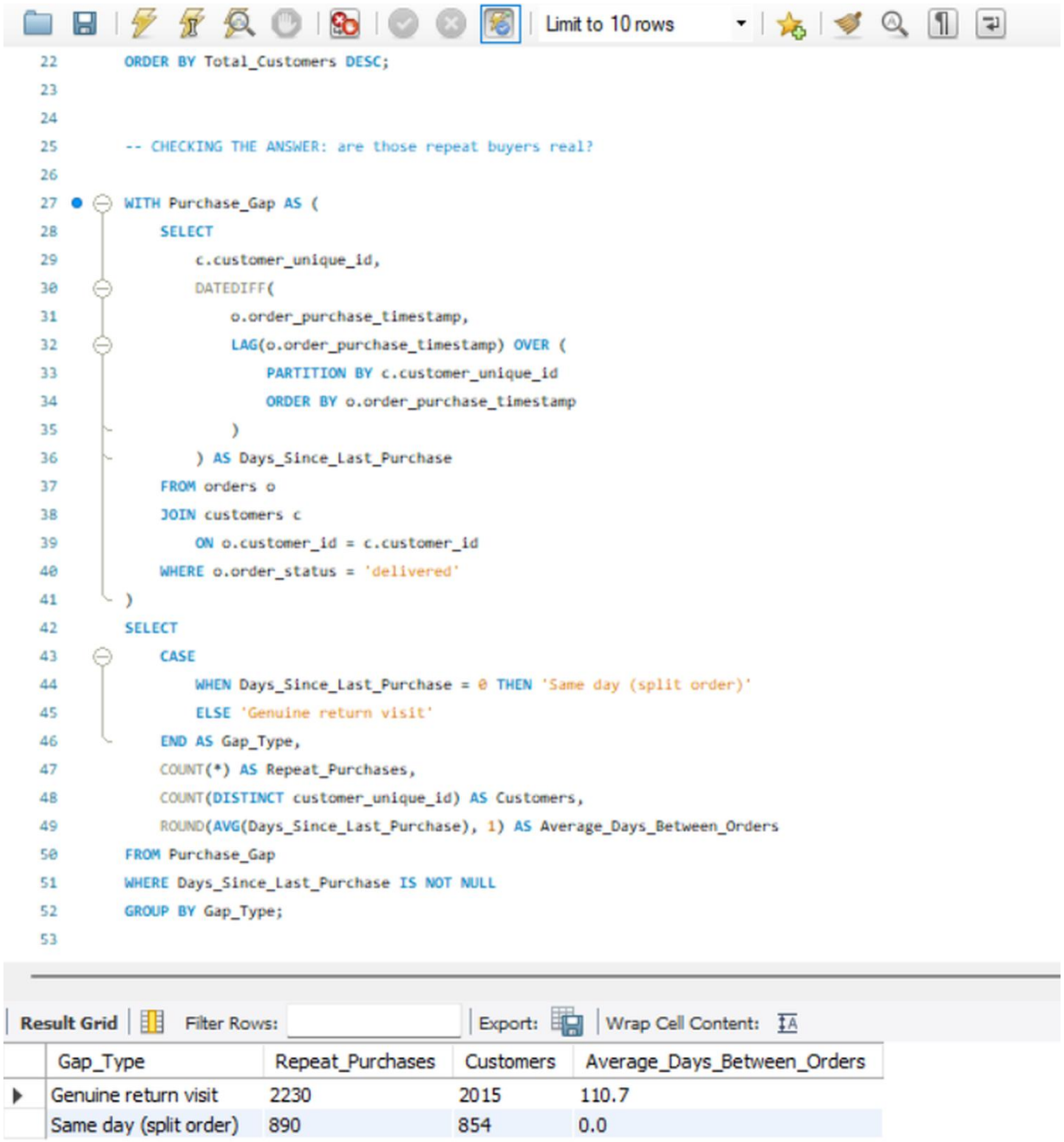
```sql
-- QUESTION 7: How many customers are repeat buyers vs one-time buyers?

WITH Customer_Orders AS (
    SELECT
        c.customer_unique_id,
        COUNT(DISTINCT o.order_id) AS Total_Orders
    FROM customers c
    JOIN orders o
        ON c.customer_id = o.customer_id
    WHERE o.order_status = 'delivered'
    GROUP BY c.customer_unique_id
)
SELECT
    CASE
        WHEN Total_Orders = 1 THEN 'One-time buyer'
        ELSE 'Repeat buyer'
    END AS Buyer_Type,
    COUNT(*) AS Total_Customers,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS Percent_Of_Customers
FROM Customer_Orders
GROUP BY Buyer_Type
ORDER BY Total_Customers DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Buyer_Type	Total_Customers	Percent_Of_Customers
One-time buyer	90557	97.00
Repeat buyer	2801	3.00

The first query returns 2,801 repeat buyers. When I looked at the rows behind that number, many second orders were placed within seconds of the first, so I checked whether those were real return visits.



```sql
ORDER BY Total_Customers DESC;


-- CHECKING THE ANSWER: are those repeat buyers real?

WITH Purchase_Gap AS (
    SELECT
        c.customer_unique_id,
        DATEDIFF(
            o.order_purchase_timestamp,
            LAG(o.order_purchase_timestamp) OVER (
                PARTITION BY c.customer_unique_id
                ORDER BY o.order_purchase_timestamp
            )
        ) AS Days_Since_Last_Purchase
    FROM orders o
    JOIN customers c
        ON o.customer_id = c.customer_id
    WHERE o.order_status = 'delivered'
)
SELECT
    CASE
        WHEN Days_Since_Last_Purchase = 0 THEN 'Same day (split order)'
        ELSE 'Genuine return visit'
    END AS Gap_Type,
    COUNT(*) AS Repeat_Purchases,
    COUNT(DISTINCT customer_unique_id) AS Customers,
    ROUND(AVG(Days_Since_Last_Purchase), 1) AS Average_Days_Between_Orders
FROM Purchase_Gap
WHERE Days_Since_Last_Purchase IS NOT NULL
GROUP BY Gap_Type;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Gap_Type	Repeat_Purchases	Customers	Average_Days_Between_Orders
Genuine return visit	2230	2015	110.7
Same day (split order)	890	854	0.0

890 were same-day split orders, not returns. These are single baskets split into separate order IDs because the items came from different sellers. The genuine figure is 2,015 customers, taking an average of 110.7 days to come back.

Question 8: What percentage of total revenue comes from the top category?

```sql
-- QUESTION 8: What percentage of total revenue comes from the top category?

WITH Category_Revenue AS (
    SELECT
        COALESCE(p.product_category_name, 'Uncategorized') AS Product_Category,
        SUM(oi.price) AS Total_Revenue
    FROM products p
    JOIN order_items oi
        ON p.product_id = oi.product_id
    JOIN orders o
        ON oi.order_id = o.order_id
    WHERE o.order_status = 'delivered'
    GROUP BY Product_Category
)
SELECT
    Product_Category,
    ROUND(Total_Revenue, 2) AS Category_Revenue,
    ROUND(SUM(Total_Revenue) OVER (), 2) AS All_Categories_Revenue,
    ROUND(Total_Revenue * 100.0 / SUM(Total_Revenue) OVER (), 2) AS Percent_Of_Total
FROM Category_Revenue
ORDER BY Category_Revenue DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Product_Category	Category_Revenue	All_Categories_Revenue	Percent_Of_Total
beleza_saude	1233131.72	13221498.11	9.33
relogios_presentes	1166176.98	13221498.11	8.82
cama_mesa_banho	1023434.76	13221498.11	7.74
esporte_lazer	954852.55	13221498.11	7.22
informatica_acessorios	888724.61	13221498.11	6.72

This query uses order_items.price, so the total shown here is product revenue and excludes freight. Revenue measured from payment records, which includes freight, totals R$15,422,462.